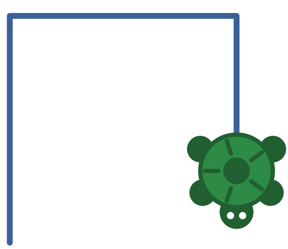
The Turtle Book

A longer book about Logo,
with a little BASIC at the end



For ________________

To write: Her name goes on the line on the cover. Decide whether this book is hers alone or shared; the kidbook names both girls.

What Is Inside

Part I.

The Turtle

CHAPTER 1

Before You Start

To write: How to open Logo from the menu ("Draw Pictures with Logo Turtle"). What appears: a grey screen, a small triangle in the middle (that is the turtle, seen from above), a house it has already drawn, and a question mark where you type. You type a line, press **Enter**, the turtle does it. Mistakes are fine and expected; the worst that happens is a message (Chapter 13). How to leave: type `bye`, or hold **Ctrl** and **Alt** and press **Backspace**.
Decide about case: this book prints commands in lowercase because it is easier to type, but the kidbook uses `FD 50`. Logo accepts both. Say so once.

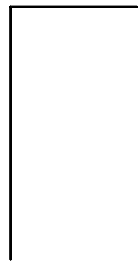
CHAPTER 2

Meet the Turtle

To write: The turtle knows four moving words: `fd` (forward), bk (back), `rt` (right), `lt` (left). Each one wants a number after it: how far, or how much to turn. The turtle always has a place and a direction it is facing; `fd` changes the place, `rt` and `lt` change the direction. A good first activity away from the computer: one person is the turtle, the other gives commands, and the turtle may only do exactly what was said.

Type these three lines, one at a time, pressing **Enter** after each:

```
fd 100
rt 90
fd 50
```



To write: Two more words: `cs` wipes the screen and puts the turtle back in the middle facing up; `home` sends it back without wiping.

Try

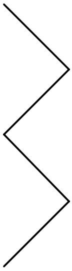
- `bk 100` — does the turtle draw when it walks backwards?
- `rt 90` four times in a row. Where is it facing now?
- Draw the letter L. Then the letter T (you will need `bk`).

CHAPTER 3

Turning

To write: What the number after `rt` means. `rt 90` is a quarter turn, the corner of a square; `rt 180` is turning to face the opposite way; `rt 360` is all the way around, so the turtle ends up facing where it started. Smaller numbers are smaller turns. `rt 45` is half of a corner. Do it with your body: face the window, `rt 90`, now you face the door.

```
rt 45
fd 50
lt 90
fd 50
rt 90
fd 50
lt 90
fd 50
```



Try

- Change every `50` to `30`. Then to `80`.
- Change `45` to `20`. What changed?
- Make the zigzag longer by typing more lines.

CHAPTER 4

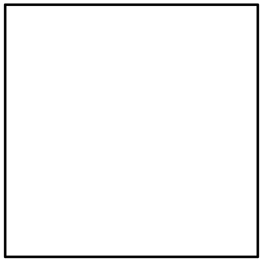
Saying It Again

Here is a square, the long way:

```
fd 100
rt 90
fd 100
rt 90
fd 100
rt 90
fd 100
rt 90
```

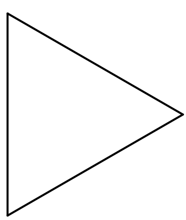
To write: Notice the same two lines typed four times. Logo has a word for “do this again”: `repeat`, then how many times, then the commands inside square brackets. The brackets say where the repeated part starts and ends.

```
repeat 4 [fd 100 rt 90]
```

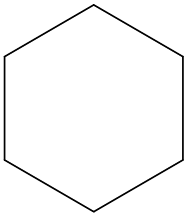


To write: The big idea in this chapter: to draw a closed shape and end up facing the way it started, the turtle turns all the way around once, which is 360. Four corners share 360, so each is 90. Three corners share 360, so each is 120. Let her check the table with a calculator or by counting up.

```
repeat 3 [fd 100 rt 120]
```



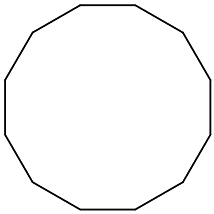
```
repeat 6 [fd 60 rt 60]
```



sides	turn	
3	120	triangle
4	90	square
5	72	pentagon
6	60	hexagon
8	45	octagon
12	30	

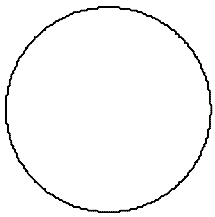
To write: Sides times turn is always 360. Invite her to guess what twelve sides will look like before running it.

```
repeat 12 [fd 30 rt 30]
```



To write: And if the turtle takes 360 tiny steps with 360 tiny turns, the corners disappear.

```
repeat 360 [fd 1 rt 1]
```



Try

- `repeat 4 [fd 100 rt 90]` but change `90` to `91`. Then `repeat 40` of that.
- `repeat 5 [fd 100 rt 144]` — a star. Why 144 and not 72?
- `repeat 360 [fd 2 rt 1]` — a bigger circle.

CHAPTER 5

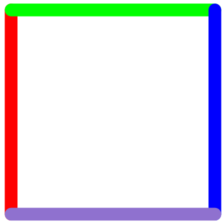
The Pen

To write: The turtle carries a pen. pu lifts it so the turtle moves without drawing; pd puts it back down. This is how you draw two things that do not touch.

```
repeat 8 [fd 15 pu fd 10 pd]
```


To write: `setpc` sets the pen colour by number and `setpensize` makes the line thicker. The two numbers in `setpensize [5 5]` are width and height; just keep them the same.

```
setpensize [5 5]
setpc 4
fd 80
rt 90
setpc 2
fd 80
rt 90
setpc 1
fd 80
rt 90
setpc 13
fd 80
rt 90
```

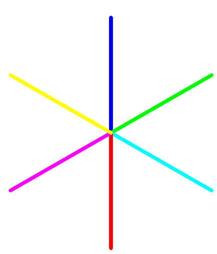


0	black	8	brown
1	blue	9	tan
2	green	10	forest green
3	light blue	11	aqua
4	red	12	salmon
5	magenta	13	purple
6	yellow	14	orange
7	white	15	grey

To write: `setbg` changes the background with the same numbers. The screen starts grey (15) with a black pen because the welcome file sets it that way.

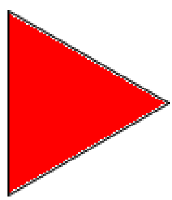
To write: Inside a `repeat`, the word `repcount` is the number of the turn Logo is on: 1 the first time, 2 the second, and so on. Using it as the pen colour gives a different colour each spoke.

```
setpensize [3 3]
repeat 6 [setpc repcount fd 100 bk 100 rt 60]
```



To write: `fill` pours colour into the closed shape the turtle is standing inside, so first draw the shape, then lift the pen and step inside. If the turtle is outside the shape, the colour goes outside.

```
repeat 3 [fd 100 rt 120]
pu
rt 30
fd 30
pd
setpc 4
fill
```



Try

- A dotted circle: `repeat 36 [fd 5 pu fd 5 pd rt 10]`
- Every colour: `repeat 16 [setpc repcount fd 20 rt 90 f`
- Fill the house that was on the screen when Logo opened.

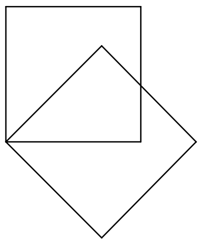
CHAPTER 6

Teaching the Turtle a New Word

To write: The turtle only knows the words it was born with, but you can teach it more. `to square` starts a lesson; the lines after it are what `square` means; `end` finishes the lesson. While you are teaching, the prompt changes from ? to > so you know Logo is listening to the lesson rather than doing things. After `end`, `square` is a word like any other.

```
to square
  repeat 4 [fd 100 rt 90]
end

square
rt 45
square
```



To write: Useful words about words: `po "square` shows the lesson back to you (the quote mark is part of it); `pots` lists every word you have taught; `erase "square` forgets one.
Words are forgotten when Logo closes. `save "mine.lg` writes them all to a file and `load "mine.lg` brings them back next time. The file lands wherever Logo was started from, which the menu keeps the same every time, so `load` finds it again.

Try

- Teach the turtle `triangle`. Then `hexagon`.
- `repeat 8 [square rt 45]`
- Teach it `box`: a square, then back to where it started, facing the same way. Check with `repeat 3 [box fd 120]`.

#

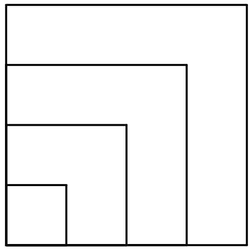
CHAPTER 7

Words That Take a Number

To write: fd takes a number; why can't square? It can. to square :size means "square wants one number, and inside the lesson we will call it size." The colon (dots) means "the number that was handed to me," not the word itself. Then square 30 and square 120 are the same lesson with different numbers.

```
to square :size
  repeat 4 [fd :size rt 90]
end

square 30
square 60
square 90
square 120
```

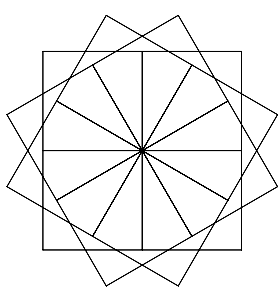


To write: A word you taught can be used inside another lesson, exactly like a built-in word. `flower` hands its own number on to `square`.

```
to square :size
  repeat 4 [fd :size rt 90]
end

to flower :size
  repeat 12 [square :size rt 30]
end

flower 80
```



Try

- `flower 30`, `flower 120`
- Change `12` and `30` in `flower` (keep them multiplying to 360, or don't, and see).
- Teach `triangle :size` and make a triangle flower.

CHAPTER 8

Building Big Things

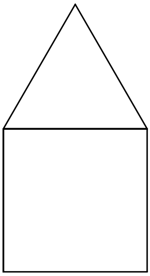
To write: Big pictures are made from small words. A house is a square with a triangle on top; the tricky part is walking the turtle to the top-left corner and tilting it 30 degrees so the roof sits flat. Trace it with a finger, line by line, saying where the turtle is. This is the house that is already on the screen when Logo opens.

```
to square :size
  repeat 4 [fd :size rt 90]
end

to triangle :size
  repeat 3 [fd :size rt 120]
end

to house :size
  square :size
  fd :size
  rt 30
  triangle :size
  lt 30
  bk :size
end

house 100
```



To write: `house` leaves the turtle exactly where it started, facing the same way. That is on purpose: it makes `house` safe to use inside a `repeat`. Walk right with the pen up, draw another.

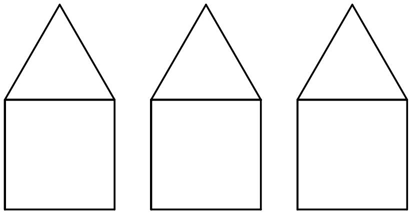
```
to square :size
  repeat 4 [fd :size rt 90]
end

to triangle :size
  repeat 3 [fd :size rt 120]
end

to house :size
  square :size
  fd :size
  rt 30
  triangle :size
  lt 30
  bk :size
end

to street
  repeat 3 [house 60 pu rt 90 fd 80 lt 90 pd]
end

street
```



💡 **Try**

- A street of five houses. (Do they fit? Make them smaller.)
- A door: a small tall rectangle. Where does the turtle have to walk first?
- A window, a chimney, a sun (`circle`) in the sky.

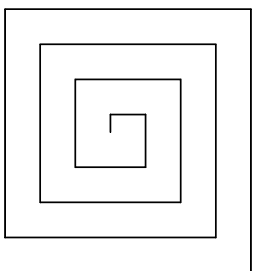
CHAPTER 9

A Word That Uses Itself

To write: The strangest and best trick. `spiral` draws one side, turns, and then calls `spiral` again with a slightly bigger number. Each square is a little bigger than the last, so the line spirals out.
The `if` line is the stop rule: once the number gets past 150, `stop` ends the lesson and the turtle rests. Ask what would happen without it: it would never stop. The way out is the **Logo** menu at the top of the window, then **Stop** (verified on this version of UCBLogo; Ctrl+C does nothing while a program runs). The menu shows Alt+S as a shortcut, still to be tried on the Pi. If all else fails, **Ctrl** + **Alt** + **Backspace** leaves Logo for the menu, as the kidbook says.

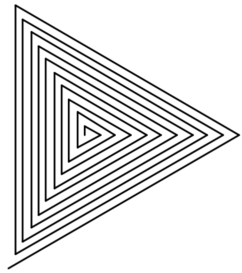
```
to spiral :size
  if :size > 150 [stop]
  fd :size
  rt 90
  spiral :size + 10
end

spiral 10
```



```
to trispiral :size
  if :size > 150 [stop]
  fd :size
  rt 120
  trispiral :size + 5
end

trispiral 5
```



Try

- In `spiral`, change `rt 90` to `rt 91`. Then `rt 89`.
- Change `+ 10` to `+ 3`.
- Change `150` to `300`.
- Make a `hexspiral` with `rt 60`.

CHAPTER 10

The Turtle Can Talk

To write: `print` writes in the text part of the screen. Words go in square brackets. Numbers can be added: `sum 2 3` or `2 + 3`, times is `*`. So Logo is also a calculator.

```
print [Hello!]
print sum 2 3
print 7 * 6
print [I am a turtle.]
```

```
Hello!
5
42
I am a turtle.
```

To write: `label` writes words on the drawing, at the turtle, facing the way the turtle faces. `setlabelheight` makes the letters bigger. Put her name in it.

```
setpensize [3 3]
setpc 1
setlabelheight 40
label [HI THERE]
```

HI THERE

Try

- `print 100 * 100`
- `rt 90 label [sideways]`
- Write your name at the four corners of a square: `repeat 4 [label [ME] fd 100 rt 90]`

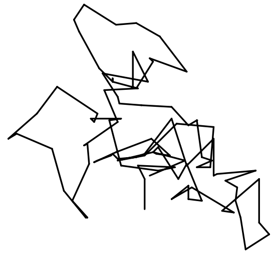
CHAPTER 11

Surprises

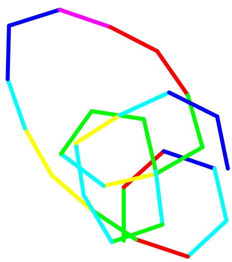
To write: `random 30` gives a number the turtle picks by itself, from 0 up to 29 (not 30; say so, she will ask). Run `print random 10` several times to see. A program with `random` in it draws a different picture every time.

```
to wander
  fd random 30
  rt random 360
end

repeat 100 [wander]
```



```
setpensize [3 3]
repeat 30 [setpc 1 + random 6 fd 40 rt random 90]
```



Try

- Run `repeat 100 [wander]` three times. Same picture?
- `repeat 20 [square random 100]`
- `repeat 50 [setpc random 16 house random 80]`

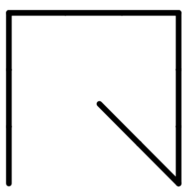
CHAPTER 12

Driving the Turtle

To write: A game. `readchar` waits for one key and hands it over; `make "key` stores it under the name `key`, and `:key` reads it back, the same colon as in `:size`. Each `if` checks for one letter. The last line calls `drive` again, so it keeps listening until you press Q. Keys: W forward, S back, A left, D right, Q quit. The picture is what the check script drove; hers will differ.

```
to drive
  make "key readchar
  if :key = "w [fd 10]
  if :key = "s [bk 10]
  if :key = "a [lt 15]
  if :key = "d [rt 15]
  if :key = "q [stop]
  drive
end

drive
```



Try

- Make W go further. Make A and D turn less.
- Add a key that lifts the pen and one that puts it down.
- Add a key that changes colour.
- Add C for `cs`.

CHAPTER 13

When Things Go Wrong

To write: The turtle never breaks; it just says it does not understand. These are the four messages she will see most. Each one is the turtle being honest about exactly what confused it.

The turtle says	What it means
`I don’t know how to squar`	A word it has not learned. Usually a spelling slip, or a lesson you have not taught yet in this session.
`not enough inputs to fd`	`fd` wanted a number and did not get one.
`You don’t say what to do with 50`	A number with no word in front of it. The turtle has 50 and no idea what for.
`size has no value`	`:size` used outside a lesson that was handed a size.

To write: Also: if the turtle will not stop, use the Logo menu at the top, then Stop. A missing `]` makes Logo wait silently for more (the prompt changes); type `]` and Enter. And the difference between `square` and `"square`: without the quote, Logo runs the word; with it, Logo means the name.

CHAPTER 14

Words the Turtle Knows

Moving

`fd 50`	forward
`bk 50`	back
`rt 90`	turn right
`lt 90`	turn left
`cs`	clear the screen, turtle to the middle
`home`	turtle to the middle, keep the drawing
`ht`	hide the turtle
`st`	show the turtle

The pen

`pu`	pen up
`pd`	pen down
`setpc 4`	pen colour
`setbg 7`	background colour
`setpensize [5 5]`	thicker pen
`fill`	pour colour into the shape you are inside

Again and again

`repeat 4 [ ... ]`	do it again
`repcount`	which time this is, inside a repeat
`to name :size`	start teaching a word
`end`	finish teaching
`po "name`	show a lesson
`pots`	list all lessons
`erase "name`	forget a lesson
`save "f.lg`	keep lessons in a file
`load "f.lg`	get them back
`if :size > 150 [stop]`	stop rule

Talking

`print [hi]`	write words
`label [hi]`	write words on the drawing
`random 30`	a number from 0 to 29
`make "key readchar`	wait for a key and remember it
`wait 60`	pause one second
`bye`	leave Logo

Part II.

A Little BASIC

CHAPTER 15

A Different Language

To write: BASIC is another computer language on the same computer ("Write BASIC Programs" on the menu). It is better at words and questions than at drawing. Two differences from Logo up front: every line of a program starts with a number, and the computer does nothing until you type `RUN`. The numbers say what order to go in; counting by tens leaves room to squeeze a line in between later. `LIST` shows the program, `NEW` throws it away.
Stopping a runaway program: **Ctrl** + **C**, as the kidbook says (PC-BASIC treats it like Ctrl+Break by default, so the welcome screen's Ctrl+Break works too).

```
10 CLS
20 PRINT "HELLO!"
30 PRINT "MY NAME IS COMPUTER."
40 PRINT 2 + 3
50 END
```

To write: Line 40: with quotes, `PRINT` shows the letters; without quotes, it works out the sum first. Same trick as Logo's `print 2 + 3`.

Try

- Add a line 45 that prints your age. Type `LIST`. Where did it go?
- `PRINT 100 * 100` with no line number. It runs right away.
- Type line 20 again with different words. Which one wins?

CHAPTER 16

The Computer Asks

To write: `INPUT` prints the question, waits for an answer, and stores it in `N$`. The dollar sign means “this box holds words”; a box for numbers has no dollar sign. Semicolons in `PRINT` glue pieces together on one line.

```
10 CLS
20 INPUT "WHAT IS YOUR NAME"; N$
30 PRINT "HELLO, "; N$; "!"
40 PRINT "NICE TO MEET YOU, "; N$
50 END
```

```
WHAT IS YOUR NAME?
```

Try

- Ask for a favourite animal too, and print both.
- Print the name ten times (the kidbook's `FOR` loop).

CHAPTER 17

Mad Libs

To write: Four questions, then a story with the answers dropped in. This is the program she will want to change most, so leave room. Note line 80: `N` is a number box, so no dollar sign, and BASIC puts a space on each side of a printed number by itself.

```
10 CLS
20 INPUT "AN ANIMAL"; A$
30 INPUT "A COLOR"; C$
40 INPUT "A FOOD"; F$
50 INPUT "A NUMBER"; N
60 PRINT
70 PRINT "ONCE THERE WAS A "; C$; " "; A$; "."
80 PRINT "IT ATE"; N; "PLATES OF "; F$; " EVERY DAY."
90 PRINT "THE END."
100 END
```

```
AN ANIMAL? CAT
A COLOR? PURPLE
A FOOD? SPAGHETTI
A NUMBER?
```

Try

- Write your own story with five blanks.
- Ask for a name and make the story about that person.

CHAPTER 18

Guess My Number

To write: Three new pieces: `RND` picks the secret number (line 30 turns a fraction into 1 to 10); `IF . . . THEN` does the rest of the line only when the test is true; `GOTO 50` jumps back to the question. Ask why the program needs `GOTO` here (it must ask again) and why line 80 does not (it is finished).

```
10 CLS
20 RANDOMIZE TIMER
30 N = INT(RND * 10) + 1
40 PRINT "I AM THINKING OF A NUMBER FROM 1 TO 10."
50 INPUT "WHAT IS YOUR GUESS"; G
60 IF G < N THEN PRINT "TOO LOW!": GOTO 50
70 IF G > N THEN PRINT "TOO HIGH!": GOTO 50
80 PRINT "YES! IT WAS "; N
90 END
```

```
I AM THINKING OF A NUMBER FROM 1 TO 10.
WHAT IS YOUR GUESS? 5
TOO LOW!
WHAT IS YOUR GUESS? 1
TOO LOW!
WHAT IS YOUR GUESS? 2
TOO LOW!
WHAT IS YOUR GUESS? 3
TOO LOW!
WHAT IS YOUR GUESS? 4
TOO LOW!
WHAT IS YOUR GUESS? 6
TOO LOW!
WHAT IS YOUR GUESS?
```

Try

- Make it 1 to 100. How many guesses do you need if you are clever?
- Count the guesses: add `T = T + 1` and print `T` at the end.
- Swap roles: you pick, the computer guesses (hard; do it together).

CHAPTER 19

Counting Down, Colours, Music

To write: `FOR I = 10 TO 1 STEP -1` counts backwards. Line 40 is a musical rest: `PLAY "P2"` plays silence for half a note, which is one second, and that is how the countdown gets its beat (a do-nothing loop is no good; this computer runs it in no time). `BEEP` at the end is the speaker. Whether the beep is audible depends on the Pi's sound, see doc/PI-CHECKLIST.txt.

```
10 CLS
20 FOR I = 10 TO 1 STEP -1
30 PRINT I
40 PLAY "P2"
50 NEXT I
60 PRINT "BLAST OFF!"
70 BEEP
80 END
```

To write: `COLOR` changes the colour of the text that comes after it, numbers 1 to 15 like Logo's pen. `COLOR 7` puts it back to normal.

```
10 CLS
20 FOR C = 1 TO 15
30 COLOR C
40 PRINT "THIS IS COLOR NUMBER"; C
50 NEXT C
60 COLOR 7
70 END
```

To write: `PLAY` takes a string of note names. A number after a note makes it longer (2 is a half note). This one is Twinkle Twinkle. **The one thing left to check on the Pi**: that PC-BASIC actually makes sound there (doc/PI-CHECKLIST.txt). If it does not, drop this section and the `BEEP` in the countdown.

```
10 PLAY "C C G G A A G2"
20 PLAY "F F E E D D C2"
```

Try

- Count down from 20. Count up.
- Print your name in every colour.
- `PLAY "C D E F G A B > C"` — a scale. What does `>` do?
- Write the next line of Twinkle Twinkle.

CHAPTER 20

Two Languages, Same Ideas

To write: A closing page. Both languages have a way to say “again” (`repeat` / `FOR`), a way to name a thing (`make` / `N =`), a way to choose (`if` / `IF THEN`), and a way to ask (`readchar` / `INPUT`). The words differ; the ideas are the same, and they are the same in the languages Daddy uses at work. What to do next: pick one of the “Try” ideas and make it yours.

My Turtle Words

My Programs

The Turtle Book
Updated September 4, 2026